

방금 설명드린 *******이벤트를 활용한 모듈 간 디커플링 및 데이터 교환 패턴*******을 팀원들과 공유하거나 나중에 다시 보실 수 있도록 깔끔하게 정리해 드릴게요.

현재 제가 직접 외부로 연결되는 공유 링크(URL)를 생성할 수는 없지만, 아래 내용을 **Markdown** 파일이나 노션(**Notion**), 사내 **Wiki**에 바로 복사해서 붙여넣으실 수 있도록 문서를 구성했습니다.

[기술 문서] Spring Event 기반 모듈 간 Request-Response 구현 가이드

1. 개요 (Background)

모놀로식 시스템 내에서 모듈 간 결합도(Coupling)를 낮추기 위해 `ApplicationEventPublisher`를 사용한다. 일반적인 이벤트는 단방향(Fire-and-Forget)이지만, 타 모듈로부터 데이터를 받아와야 하는 경우 **Consumer Callback** 패턴을 사용하여 의존성을 완전히 제거할 수 있다.

2. 아키텍처 구조

- **Module A (요청자):** 이벤트를 발행하고, 응답이 올 때 수행할 동작을 `Callback` 함수로 정의하여 전달한다.
- **Module B (처리자):** 이벤트를 구독하여 로직을 수행하고, 이벤트 객체에 담긴 `Callback`을 실행하여 데이터를 돌려준다.
- **Common Module:** 두 모듈이 공유하는 `Event` 객체만 존재하며, 서로의 서비스나 인터페이스에는 의존하지 않는다.

3. 상세 구현 코드

① 공유 이벤트 객체 (Common)

응답 데이터를 담을 `Consumer` 인터페이스를 필드로 가집니다.

Java

```
import java.util.function.Consumer;
```

```
/**
```

```
 * 데이터를 요청하기 위한 공통 이벤트 객체
```

```
 */
```

```
public class DataRequestEvent<T> {
```

```

private final String requestId;    // 요청 식별자
private final Consumer<T> callback; // 응답을 처리할 콜백 함수

public DataRequestEvent(String requestId, Consumer<T> callback) {
    this.requestId = requestId;
    this.callback = callback;
}

// 데이터를 채워주는 시점에 호출
public void sendResponse(T data) {
    if (callback != null) {
        callback.accept(data);
    }
}

public String getRequestId() { return requestId; }
}

```

② 데이터를 요청하는 쪽 (Module A)

상대 모듈의 존재를 몰라도 되며, 오직 이벤트만 발행합니다.

Java

```

@Service
@RequiredArgsConstructor
public class OrderService {
    private final ApplicationEventPublisher eventPublisher;

    public void checkProductStock(String productId) {
        // 1. 이벤트 생성 및 콜백 정의
        DataRequestEvent<Integer> event = new DataRequestEvent<>(productId, stock -> {
            // 응답을 받았을 때의 로직을 여기에 작성
            System.out.println("상품 " + productId + "의 현재 재고: " + stock);
        });

        // 2. 이벤트 발행
        eventPublisher.publishEvent(event);
    }
}

```

③ 데이터를 제공하는 쪽 (Module B)

요청자의 정보를 몰라도 되며, 들어온 이벤트의 콜백만 실행합니다.

Java

```
@Component
public class InventoryService {

    @EventListener
    public void handleStockInquiry(DataRequestEvent<Integer> event) {
        // 1. 비즈니스 로직 수행 (예: DB 조회)
        int currentStock = 50;

        // 2. 콜백을 통해 데이터 전달
        event.sendResponse(currentStock);
    }
}
```

4. 이 방식의 장점 (Benefits)

1. 완전한 디커플링: 모듈 A와 B 사이에 컴파일 타임 의존성이 0입니다. (서로를 import 하지 않음)
2. 유지보수성: 데이터를 제공하는 모듈이 바뀌거나 추가되어도 요청부의 코드를 수정할 필요가 없습니다.
3. 동기적 흐름: 기본적으로 스프링 이벤트는 동기 방식으로 동작하므로, `publishEvent`가 끝나는 시점에 결과값이 보장됩니다.

5. 주의사항

- **@Async** 사용 시 주의: 리스너에 **@Async**가 붙으면 별도 스레드에서 동작하므로, 요청 스레드가 결과를 기다리지 않고 지나가 버립니다. 비동기 환경에서는 **CompletableFuture** 등을 활용한 추가 처리가 필요합니다.
- 리스너 부재: 만약 이벤트를 처리할 리스너가 하나도 없다면 콜백이 실행되지 않으므로 예외 케이스 처리가 필요할 수 있습니다.

이 내용을 복사해서 팀원분들에게 전달하시면 "왜 인터페이스 주입보다 이게 더 유연한지"

설명하기 훨씬 수월하실 거예요.

혹시 이 내용을 이미지 파일로 렌더링해서 드리는 게 편하실까요, 아니면 이대로 복사해서 사용하시겠어요? 가독성을 위해 추가로 조정하고 싶은 부분이 있다면 말씀해 주세요!